Forward Deployed Engineer (FDE) Assessment Tasks

Target Roles: Forward Deployed Engineer (FDE) / AI Integration Engineer
Core Focus Areas: Model Context Protocol (MCP) Servers, MCP Gateways, LLM Gateways, Security Guardrails, & System Integration

Overview

This assessment consists of 5 practical technical tasks designed to evaluate candidates on building MCP servers, implementing LLM & MCP security proxy gateways, handling stream guardrails, managing model routing resilience, and troubleshooting zero-trust network deployments.

Task 1: Build a Custom MCP Server with Strict Validation & Transport Handling

Format: Take-Home / 60-min Live Coding

Problem Statement

Write a runnable MCP server (in TypeScript or Python) using the official SDK (@modelcontextprotocol/sdk or mcp) that exposes two tools:

1. get_customer_record (Input: customer_id string formatted as CUST-XXXXX).
2. trigger_refund (Inputs: customer_id, amount positive float, reason string with minimum length of 10).

Requirements

- Enforce strict input schema validation (using Zod or Pydantic). Reject invalid formats with standard MCP JSON-RPC error codes.
- Connect via stdio transport. Ensure stdout is strictly reserved for JSON-RPC messages and system/debug logs write exclusively to stderr.

Evaluation Criteria (What to Score)

- **STDIO Isolation:** Verification that stdout is pure JSON-RPC without accidental console.log or print statements.
- **Protocol Compliance:** Standard JSON-RPC error mapping and execution flow.
- **Validation:** Robust schema definitions and edge-case handling for malformed input fields.

Task 2: Implement an MCP Security Gateway Proxy

(Tool Filtering & Auth)

Format: Take-Home Coding Challenge

Problem Statement

Build a lightweight HTTP/JSON-RPC reverse proxy (in Node.js or Python) that acts as an **MCP Gateway** sitting between an AI agent client and a downstream mock MCP server.

Requirements

- Read an incoming Bearer <token> HTTP header and extract the user's role (e.g., admin or viewer).
- Inspect the incoming MCP JSON-RPC payload:
    - If method is tools/list, forward the request transparently to the downstream server.
    - If method is tools/call, inspect params.name. If params.name starts with admin_ (e.g., admin_reset_key), verify that the token role is admin. If not authorized, intercept and return a JSON-RPC Error (-32001: Unauthorized Tool Call) directly without calling the downstream server.

Evaluation Criteria (What to Score)

- Parsing JSON-RPC wire format structures correctly.
- Proxy middleware construction and HTTP request/response forwarding.
- Fine-grained, method-level authorization logic and clean error handling.

Task 3: Implement an LLM Gateway Streaming Guardrail (PII Redaction)

Format: Coding Assessment

Problem Statement

Implement an **LLM Gateway** proxy endpoint that routes text generation requests to an LLM provider and streams the response back to the client.

Requirements

- Intercept the returning response chunk stream in real time.
- Parse incoming stream deltas to detect and redact sensitive patterns (such as Emails, SSNs, or credit card numbers), replacing them with [REDACTED].
- Ensure the stream stays responsive without accumulating the full response in memory, minimizing Time To First Token (TTFT) and overall stream latency.

Evaluation Criteria (What to Score)

- Efficient asynchronous stream chunking and buffer state management.
- Performant string/regex pattern matching over partial text streams.
- Memory efficiency and low-latency proxying design.

Task 4: Build a Rate-Limiting & Model Fallback Router for LLM Gateways

Format: Architectural Coding Task

Problem Statement

Write a resilient model-routing module for an LLM Gateway that accepts incoming completion requests.

Requirements

- Implement a token-aware sliding window rate limiter (e.g., maximum 50,000 tokens/minute per tenant API key).
- If the primary model endpoint returns a 429 Too Many Requests status or times out after 3000ms, automatically failover to a secondary backup model provider.
- Ensure error responses return a standardized gateway error payload without leaking raw upstream stack traces or internal implementation details.

Evaluation Criteria (What to Score)

- Async concurrency handling and timeout race conditions.
- Accurate rate-limiter state eviction and token tracking logic.
- Graceful fallback mechanics and standardized error sanitization.

Task 5: System Design & Debugging Scenario — Broken MCP Connection in Corporate Network

Format: Technical Scenario Writing / Oral Technical Assessment

Problem Statement

"An enterprise client deploys our LLM Gateway and MCP Gateway inside a zero-trust corporate network behind a strict egress proxy. Agents experience intermittent connection drops during multi-turn tool execution, resulting in hanging requests."

Requirements

Write out a step-by-step technical isolation and debugging strategy covering:

1. Commands and diagnostic tools you would run on-site or during a live debugging session (e.g., curl, tcpdump, proxy/TLS log inspection).

2. How to systematically determine whether the failure point is at the mTLS handshake level, SSE session timeout, or LLM context truncation.
3. How you would redesign the transport layer or connection pooling mechanism to ensure connection resilience.

Evaluation Criteria (What to Score)

- Enterprise network debugging instincts and practical Linux CLI proficiency.
- Systematic troubleshooting workflow under high customer visibility.
- Architectural clarity when proposing long-term technical remediations.